Prolog Programs 20 - 25

Source Code, Input (Queries) and Output

20. PLANETS DB
21. Towers of Hanoi
22. Bird Can Fly or Not
23. Family Tree
24. Dieting System based on Disease
25. Monkey Banana Problem

Program 20: Prolog Program for PLANETS DB

Aim: Store facts about the planets of the Solar System and answer queries on them.

Source Code

```
% planet(Name, OrderFromSun, DiameterKm, Type)
planet(mercury, 1,   4879, terrestrial).
planet(venus,   2,  12104, terrestrial).
planet(earth,   3,  12756, terrestrial).
planet(mars,    4,   6792, terrestrial).
planet(jupiter, 5, 142984, gas_giant).
planet(saturn,  6, 120536, gas_giant).
planet(uranus,  7,  51118, ice_giant).
planet(neptune, 8,  49528, ice_giant).

bigger_than(X, Y) :-
    planet(X, _, DX, _),
    planet(Y, _, DY, _),
    DX > DY.

list_planets :-
    planet(N, O, D, T),
    format('~w. ~w  Diameter: ~w km  Type: ~w~n', [O, N, D, T]),
    fail.
list_planets.
```

Input (Queries)

```
?- list_planets.
?- planet(earth, Pos, Dia, Type).
?- planet(P, _, _, gas_giant).
?- bigger_than(jupiter, earth).
?- findall(P, planet(P, _, _, terrestrial), L).
?- planet(P, 3, _, _).
```

Output

```
?- list_planets.
1. mercury  Diameter: 4879 km  Type: terrestrial
2. venus  Diameter: 12104 km  Type: terrestrial
3. earth  Diameter: 12756 km  Type: terrestrial
4. mars  Diameter: 6792 km  Type: terrestrial
5. jupiter  Diameter: 142984 km  Type: gas_giant
6. saturn  Diameter: 120536 km  Type: gas_giant
7. uranus  Diameter: 51118 km  Type: ice_giant
8. neptune  Diameter: 49528 km  Type: ice_giant
true.

?- planet(earth, Pos, Dia, Type).
Pos = 3,
Dia = 12756,
Type = terrestrial.

?- planet(P, _, _, gas_giant).
P = jupiter ;
P = saturn.

?- bigger_than(jupiter, earth).
true.

?- findall(P, planet(P, _, _, terrestrial), L).
L = [mercury, venus, earth, mars].

?- planet(P, 3, _, _).
P = earth.
```

Program 21: Prolog Program to implement Towers of Hanoi

Aim: Move N disks from the source peg to the destination peg using a helper peg. Only one disk is moved at a time and a larger disk is never placed on a smaller one.

Source Code

```
hanoi(0, _, _, _) :- !.
hanoi(N, From, To, Via) :-
    N1 is N - 1,
    hanoi(N1, From, Via, To),
    write('Move disk '), write(N),
    write(' from '), write(From),
    write(' to '), write(To), nl,
    hanoi(N1, Via, To, From).
```

Input (Queries)

```
?- hanoi(3, left, right, center).
?- hanoi(2, a, c, b).
```

Output

```
?- hanoi(3, left, right, center).
Move disk 1 from left to right
Move disk 2 from left to center
Move disk 1 from right to center
Move disk 3 from left to right
Move disk 1 from center to left
Move disk 2 from center to right
Move disk 1 from left to right
true.

?- hanoi(2, a, c, b).
Move disk 1 from a to b
Move disk 2 from a to c
Move disk 1 from b to c
true.

Total moves for N disks = 2^N - 1  (N = 3 gives 7 moves).
```

Program 22: Prolog Program to check whether a particular bird can fly or not

Aim: A bird can fly unless it is listed as a non-flying bird. Queries are incorporated to test individual birds and to list all flying birds.

Source Code

```
bird(sparrow).
bird(eagle).
bird(parrot).
bird(penguin).
bird(ostrich).
bird(kiwi).

cannot_fly(penguin).
cannot_fly(ostrich).
cannot_fly(kiwi).

can_fly(X) :- bird(X), \+ cannot_fly(X).

check(B) :-
    bird(B),
    (   can_fly(B)
    ->  format('~w can fly.~n', [B])
    ;   format('~w cannot fly.~n', [B])
    ).
check(B) :-
    \+ bird(B),
    format('~w is not in the bird database.~n', [B]).
```

Input (Queries)

```
?- check(sparrow).
?- check(penguin).
?- check(dog).
?- can_fly(eagle).
?- can_fly(kiwi).
?- can_fly(X).
?- cannot_fly(X).
```

Output

```
?- check(sparrow).
sparrow can fly.
true.

?- check(penguin).
penguin cannot fly.
true.

?- check(dog).
dog is not in the bird database.
true.

?- can_fly(eagle).
true.

?- can_fly(kiwi).
false.

?- can_fly(X).
X = sparrow ;
X = eagle ;
X = parrot ;
false.

?- cannot_fly(X).
X = penguin ;
X = ostrich ;
X = kiwi.
```

Program 23: Prolog Program to implement Family Tree

Aim: Facts describe parent relationships and gender; rules derive father, mother, grandparent, sibling and ancestor relations.

Source Code

```
% Family: tom + liz -> bob, ann ;  bob + pam -> jim, sue
male(tom).     male(bob).     male(jim).
female(liz).   female(ann).   female(pam).   female(sue).

parent(tom, bob).   parent(tom, ann).
parent(liz, bob).   parent(liz, ann).
parent(bob, jim).   parent(bob, sue).
parent(pam, jim).   parent(pam, sue).

father(X, Y)      :- parent(X, Y), male(X).
mother(X, Y)      :- parent(X, Y), female(X).
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
sibling(X, Y)     :- father(F, X), father(F, Y),
                     mother(M, X), mother(M, Y), X \= Y.
ancestor(X, Y)    :- parent(X, Y).
ancestor(X, Y)    :- parent(X, Z), ancestor(Z, Y).
```

Input (Queries)

```
?- father(tom, X).
?- mother(pam, jim).
?- grandparent(tom, X).
?- sibling(jim, X).
?- ancestor(tom, jim).
?- findall(X, ancestor(X, jim), L).
```

Output

```
?- father(tom, X).
X = bob ;
X = ann.

?- mother(pam, jim).
true.

?- grandparent(tom, X).
X = jim ;
X = sue.

?- sibling(jim, X).
X = sue.

?- ancestor(tom, jim).
true.

?- findall(X, ancestor(X, jim), L).
L = [bob, pam, tom, liz].
```

Program 24: Prolog Program to suggest Dieting System based on Disease

Aim: The system stores foods to eat and to avoid for common diseases and suggests a diet for the disease entered. (For learning purposes only, not medical advice.)

Source Code

```
% diet(Disease, FoodsToEat, FoodsToAvoid)
diet(diabetes,
     'whole grains, green vegetables, lean protein',
     'sugar, sweets, white bread').
diet(hypertension,
     'fruits, vegetables, low-salt meals, banana',
     'pickles, salty snacks, processed food').
diet(obesity,
     'salads, fruits, high-fibre low-calorie food',
     'fried food, fast food, sugary drinks').
diet(anemia,
     'spinach, beetroot, dates, citrus fruits',
     'tea or coffee with meals').
diet(heart_disease,
     'oats, nuts, fish, vegetables',
     'red meat, butter, fried food').
diet(typhoid,
     'boiled rice, banana, soft food, plenty of fluids',
     'spicy and oily food').

suggest(D) :-
    diet(D, Eat, Avoid),
    format('Disease : ~w~n', [D]),
    format('Eat     : ~w~n', [Eat]),
    format('Avoid   : ~w~n', [Avoid]).
suggest(D) :-
    \+ diet(D, _, _),
    format('No diet information available for ~w.~n', [D]).
```

Input (Queries)

```
?- suggest(diabetes).
?- suggest(anemia).
?- suggest(obesity).
?- suggest(cold).
```

Output

```
?- suggest(diabetes).
Disease : diabetes
Eat     : whole grains, green vegetables, lean protein
Avoid   : sugar, sweets, white bread
true.

?- suggest(anemia).
Disease : anemia
Eat     : spinach, beetroot, dates, citrus fruits
Avoid   : tea or coffee with meals
true.

?- suggest(obesity).
Disease : obesity
Eat     : salads, fruits, high-fibre low-calorie food
Avoid   : fried food, fast food, sugary drinks
true.

?- suggest(cold).
No diet information available for cold.
true.
```

Program 25: Prolog Program to implement Monkey Banana Problem

Aim: A monkey is at the door, a box is at the window and a banana hangs from the ceiling in the middle of the room. The program finds the shortest sequence of actions for the monkey to get the banana.

Source Code

```
% state(MonkeyPos, MonkeyOn, BoxPos, HasBanana)
loc(atdoor).  loc(atwindow).  loc(middle).

% walk to another place
move(state(P, onfloor, B, H), walk(P, P2), state(P2, onfloor, B, H)) :-
    loc(P2), P2 \= P.
% push the box to another place
move(state(P, onfloor, P, H), push(P, P2), state(P2, onfloor, P2, H)) :-
    loc(P2), P2 \= P.
% climb on the box
move(state(P, onfloor, P, H), climb, state(P, onbox, P, H)).
% grasp the banana (banana is in the middle)
move(state(middle, onbox, middle, hasnot), grasp,
     state(middle, onbox, middle, has)).

plan(state(_, _, _, has), []).
plan(S1, [M|Ms]) :- move(S1, M, S2), plan(S2, Ms).

start(state(atdoor, onfloor, atwindow, hasnot)).

solve :-
    start(S),
    length(Plan, _),          % try shortest plans first
    plan(S, Plan), !,
    write_steps(Plan, 1).

write_steps([], _).
write_steps([M|Ms], N) :-
    format('Step ~w: ~w~n', [N, M]),
    N1 is N + 1,
    write_steps(Ms, N1).
```

Input (Queries)

```
?- solve.
?- start(S), length(P, _), plan(S, P), !.
```

Output

```
?- solve.
Step 1: walk(atdoor,atwindow)
Step 2: push(atwindow,middle)
Step 3: climb
Step 4: grasp
true.

?- start(S), length(P, _), plan(S, P), !.
S = state(atdoor, onfloor, atwindow, hasnot),
P = [walk(atdoor, atwindow), push(atwindow, middle), climb, grasp].
```